BİL4010

Paralel programlama

9.Hafta

MPI Sıralama

İçerik

- Sıralama Algoritmaları
  - Kabarcık sıralaması (Bubble-sort algorithm)
  - Tek-çift geçişli sıralama algoritması (Odd-Even sort)
- Paralel sıralama algoritması

Sıralama Algoritmaları

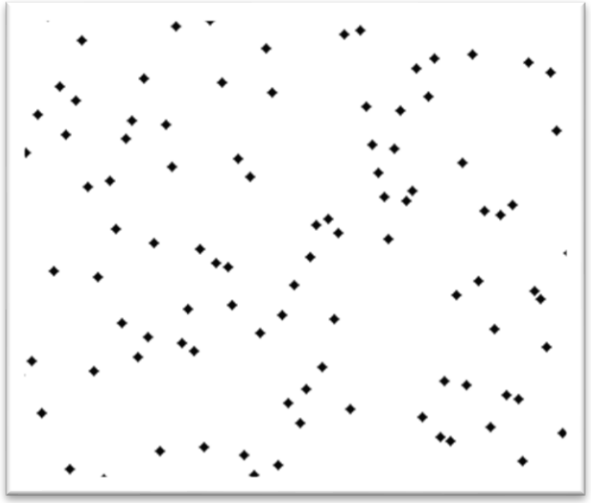
- Sıralama Algoritmaları verilen bir listenin elemanlarını belirli bir sıraya sokan algoritmadır.
- En çok kullanılan sıralama türleri, sayı büyüklüğüne göre sıralama ve alfabetik sıralamadır.
- En bilinen Algoritmalar:
  - Kabarcık Sıralaması (Bubble Sort)
  - Birleşmeli Sıralama (Merge Sort)
  - Hızlı Sıralama (Quick Sort)
  - …

Kabarcık Sıralama Algoritması

```
void Bubble_sort(
        int  a[]  /* in/out */,
        int  n    /* in     */) {
    int list_length, i, temp;

    for (list_length = n; list_length >= 2; list_length--)
        for (i = 0; i < list_length-1; i++)
            if (a[i] > a[i+1]) {
                temp = a[i];
                a[i] = a[i+1];
                a[i+1] = temp;
            }

}  /* Bubble_sort */
```



Tek-çift Geçişli Sıralama Algoritması

- Bir dizi evreden oluşur. (Tek indis ve çift indis evresi)
- Çift indis evresinde karşılaştırma ve yer değiştirme yapar:

$$(a[0], a[1]), (a[2], a[3]), (a[4], a[5]), \ldots$$

- Tek indis evresinde karşılaştırma ve yer değiştirme yapar:

$$(a[1], a[2]), (a[3], a[4]), (a[5], a[6]), \ldots.$$

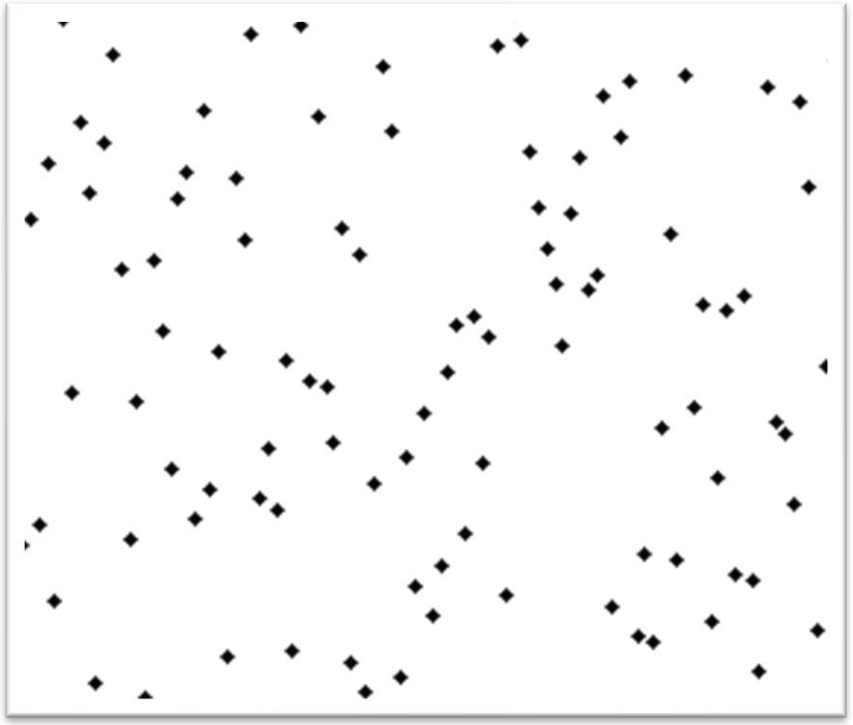
Örn: Tek-çift Geçişli Sıralama

- Dizimizin başlangıç değeri: 5, 9, 4, 3 olsun N=4
- Çift fazında (5,9) ve (4,3) karşılaştırılır ve yer değiştirme uygulanır
  - Dizi : 5,9,3,4 oldu
- Tek Fazında (9,3) karşılaştırılır ve yer değiştirme uygulanır
  - Dizi : 5,3,9,4 oldu
- Çift fazında (5,3) ve (9,4) karşılaştırılır ve yer değiştirme uygulanır
  - Dizi : 3,5,4,9 oldu
- Tek Fazında (5,4) karşılaştırılır ve yer değiştirme uygulanır
  - Dizi : 3,4,5,9 oldu
- Tek-çift fazları N defa tekrarlandığında dizi sıralanmış olur.

Tek-çift Geçişli Sıralama

```
void Odd_even_sort(
        int  a[]  /* in/out */,
        int  n    /* in     */) {
    int phase, i, temp;

    for (phase = 0; phase < n; phase++)
        if (phase % 2 == 0) { /* Even phase */
            for (i = 1; i < n; i += 2)
                if (a[i-1] > a[i]) {
                    temp = a[i];
                    a[i] = a[i-1];
                    a[i-1] = temp;
                }
        } else { /* Odd phase */
            for (i = 1; i < n-1; i += 2)
                if (a[i] > a[i+1]) {
                    temp = a[i];
                    a[i] = a[i+1];
                    a[i+1] = temp;
                }
        }
}  /* Odd_even_sort */
```



Paralel Sıralama Algoritması

- Paralel bir ortamda sıralama, çoğu zaman tek bir işlemde tüm verilere sahip olmayacağımız gerçeğinden dolayı karmaşıktır.
- Bazı durumlarda, veri miktarı tek bir makinenin belleğine sığmaz.
- Bunu çözmek için verilerimizi aşağıdaki kurallara göre sıralayabiliriz:
  - Her işlemdeki tüm veriler yerel olarak sıralanır.
  - i < j ise, işlem i üzerinde depolanan tüm veriler işlem j üzerinde depolanan tüm verilerden küçük olmalıdır.
- Paralel sıralama algoritmamızın amacı, bu iki kuralı doğru yapmaktır.
- Böylece, tüm verilerin herhangi bir işleme yüklenmesine gerek kalmadan sıralama yapılabilir.

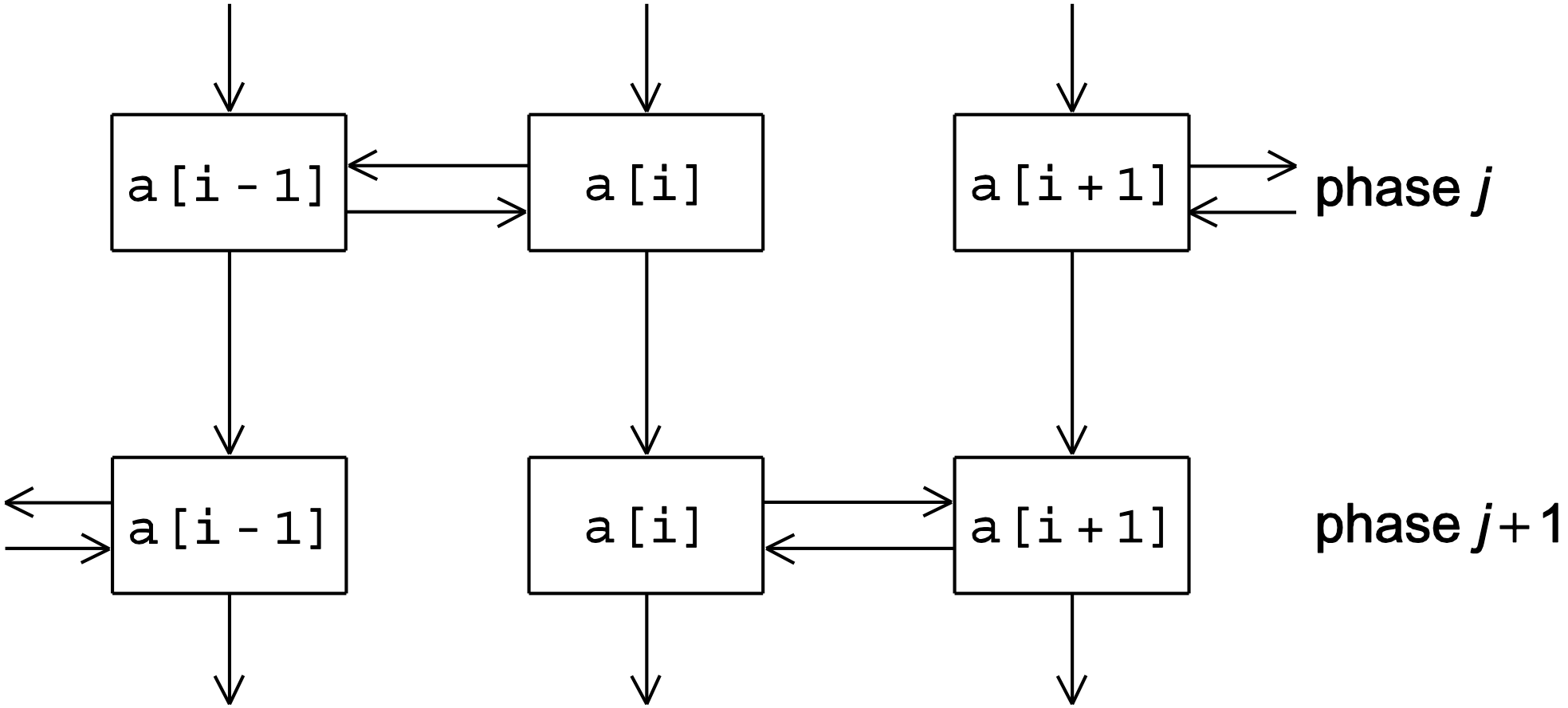
Paralel Sıralama Algoritması

- Bunu yapmak için, tekli-çift transpozisyon algoritmasının birden çok işleme genişletilmiş versiyonunu kullanacağız.
- Bu senaryoda, her işlem kendine ait verileri saklar.
- Her aşamada, her işlem kendi verilerini sıralamaya devam eder.
- Ardından, verilerimizi içinde olduğumuz aşamaya dayalı olarak sol ya da sağ işlem ile paylaşırız.

Paralel Tek-çift Transpozisyon Sıralama Algoritması

1. Her Aşamada:
   1. Yerel veriyi sırala,
   2. Aşama numarasına ve rütbesine (Rank) dayanarak ortağımızı bulun.
   3. Eğer bir ortak varsa:
      1. Sıralı veriyi ortağa gönder.
      2. Ortağının sıralı verisini al.
      3. Rütben daha küçükse kendi ve ortağının verilerinin küçük olanlarını al.
      4. Eğer, rütben daha büyükse kendi ve ortağının verilerinin büyük olanlarını al.

Tek-çift Transpozisyon Sıralama için Görevler Arası İletişim



Her Aşamada Ortak Seçim İşlemi (Pseudo-code)

```
if (phase % 2 == 0)          /* Even phase */
   if (my_rank % 2 != 0)        /* Odd rank */
      partner = my_rank - 1;
   else                         /* Even rank */
      partner = my_rank + 1;
else                         /* Odd phase */
   if (my_rank % 2 != 0)        /* Odd rank */
      partner = my_rank + 1;
   else                         /* Even rank */
      partner = my_rank - 1;
if (partner == -1 || partner == comm_sz)
   partner = MPI_PROC_NULL;
```

Ortaklar Arası Verinin Dağıtımı (Pseudo-code)

```
Sort local keys;
for (phase = 0; phase < comm_sz; phase++) {
   partner = Compute_partner(phase, my_rank);
   if (I'm not idle) {
      Send my keys to partner;
      Receive keys from partner;
      if (my_rank < partner)
         Keep smaller keys;
      else
         Keep larger keys;
   }
}
```

Paralel Tek-çift Transpozisyon Sıralama Örneği

Time	Process			
	0	1	2	3
Start	15, 11, 9, 16	3, 14, 8, 7	4, 6, 12, 10	5, 2, 13, 1
After Local Sort	9, 11, 15, 16	3, 7, 8, 14	4, 6, 10, 12	1, 2, 5, 13
After Phase 0	3, 7, 8, 9	11, 14, 15, 16	1, 2, 4, 5	6, 10, 12, 13
After Phase 1	3, 7, 8, 9	1, 2, 4, 5	11, 14, 15, 16	6, 10, 12, 13
After Phase 2	1, 2, 3, 4	5, 7, 8, 9	6, 10, 11, 12	13, 14, 15, 16
After Phase 3	1, 2, 3, 4	5, 6, 7, 8	9, 10, 11, 12	13, 14, 15, 16

Teşekkürler.

Sorular...